

ASSIGNMENT 2 – WIRESHARK/PHISHING

WIRESHARK

Question 1: what did I do?

I conducted a small survey by setting up an experiment in a local apartment to examine network activity. Using Wireshark, a widely used network protocol analyzer, I observed which networks and protocols were detectable in that environment. I also examined my own wireless network to test its range. Although my network extended beyond my home, it remained protected and was not accessible from outside.

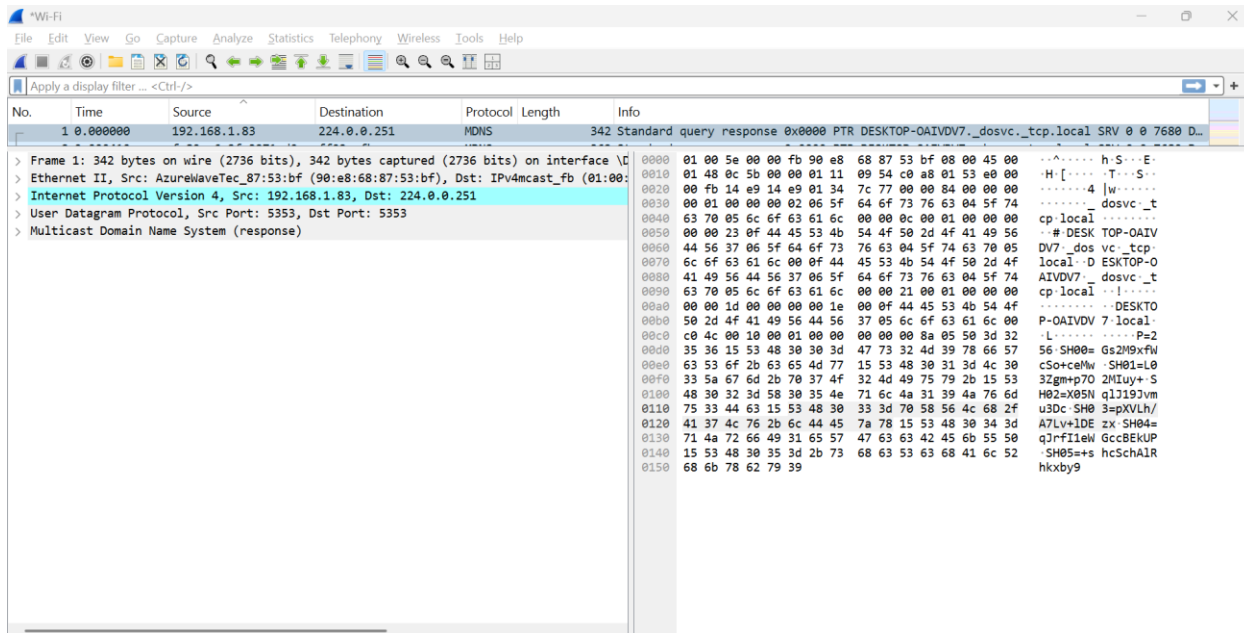
Wireshark functions as a powerful tool for analyzing network traffic. It works by capturing packets in real time and providing detailed information about their contents. By placing the network interface (such as Wi-Fi or Ethernet) into “unfiltered mode”, Wireshark is able to capture all packets passing through the interface—not just those addressed to the host computer. Once captured, the packets are decoded according to their respective protocols (e.g., TCP, UDP, DNS) and displayed in a structured, human-readable format. This makes it possible to analyze network behavior and identify patterns, anomalies, or potential security issues.

Question 2: what were the results?

After running Wireshark for a set period, I observed that it had captured roughly 25,000 packets. At that stage, I stopped the capture to begin reviewing the data. Within the collected packets, I identified a range of accessible protocols, including common ones such as TCP, UDP, ARP, and mDNS, among others.

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	192.168.1.1	224.0.0.251	ICMPv6	32	Neighbor Solicitation for fe80::1234:5678:9abc::1
2	0.000010	fe80::1234:5678:9abc::1	ff02::1	mDNS	144	Standard query response 0x0000 PTR DESKTOP-GAIVDV7._dsvnc._tcp.local SRV 0 0 7680 DESKTOP-GAIVDV7.local TXT
3	0.000078	192.168.1.83	224.0.0.251	mDNS	93	Standard query 0x0000 ANY DESKTOP-GAIVDV7._dsvnc._tcp.local, "QI" question
4	0.000078	fe80::1234:5678:9abc::1	ff02::1	mDNS	113	Standard query 0x0000 ANY DESKTOP-GAIVDV7._dsvnc._tcp.local, "QI" question
5	0.136746	2000:1700:8af0:9500::	2000:1404:a00:5::17	UDP	128	58567 → 443 Len=1224
6	0.136845	2000:1700:8af0:9500::	2000:1404:a00:5::17	UDP	128	58567 → 443 Len=1230
7	0.136875	2000:1700:8af0:9500::	2000:1404:a00:5::17	UDP	128	58567 → 443 Len=1230
8	0.136902	2000:1700:8af0:9500::	2000:1404:a00:5::17	UDP	128	58567 → 443 Len=1230
9	0.136944	2000:1700:8af0:9500::	2000:1404:a00:5::17	UDP	90	58567 → 443 Len=405
10	0.140871	2000:1404:a00:5::17	2000:1700:8af0:9500::	UDP	90	443 → 58567 Len=20
11	0.140871	2000:1404:a00:5::17	2000:1700:8af0:9500::	UDP	86	443 → 58567 Len=24
12	0.175905	2000:1404:a00:5::17	2000:1700:8af0:9500::	UDP	87	443 → 58567 Len=25
13	0.176234	2000:1700:8af0:9500::	2000:1404:a00:5::17	UDP	94	58567 → 443 Len=32
14	0.199734	2000:1404:a00:5::17	2000:1700:8af0:9500::	UDP	220	443 → 58567 Len=150
15	0.199734	2000:1404:a00:5::17	2000:1700:8af0:9500::	UDP	83	443 → 58567 Len=21
16	0.200000	2000:1700:8af0:9500::	2000:1404:a00:5::17	UDP	90	58567 → 443 Len=36
17	0.240716	2000:1404:a00:5::17	2000:1700:8af0:9500::	UDP	87	443 → 58567 Len=25
18	0.252374	192.168.1.83	224.0.0.251	mDNS	93	Standard query 0x0000 ANY DESKTOP-GAIVDV7._dsvnc._tcp.local, "QI" question
19	0.252374	fe80::1234:5678:9abc::1	ff02::1	mDNS	113	Standard query 0x0000 ANY DESKTOP-GAIVDV7._dsvnc._tcp.local, "QI" question
20	0.433711	192.168.1.83	192.168.1.65	TCP	164	57034 → 8009 [PSH, ACK] Seq=1 Ack=1 Win=252 Len=110 [TCP PDU reassembled in 238]
21	0.440876	192.168.1.65	192.168.1.83	TCP	164	8009 → 57034 [PSH, ACK] Seq=1 Ack=111 Win=434 Len=110 [TCP PDU reassembled in 239]
22	0.404427	192.168.1.83	192.168.1.65	TCP	54	57034 → 8009 [ACK] Seq=111 Ack=111 Win=252 Len=0
23	0.537208	192.168.1.83	224.0.0.251	mDNS	93	Standard query 0x0000 ANY DESKTOP-GAIVDV7._dsvnc._tcp.local, "QI" question
24	0.517614	fe80::1234:5678:9abc::1	ff02::1	mDNS	113	Standard query 0x0000 ANY DESKTOP-GAIVDV7._dsvnc._tcp.local, "QI" question
25	0.760014	192.168.1.83	224.0.0.251	mDNS	491	Standard query response 0x0000 PTR, cache flush DESKTOP-GAIVDV7._dsvnc._tcp.local SRV, cache flush 0 0 7680 DESKTOP-GAIVDV7.local TXT, cache flush A, cache flush 192.168.1.1...
26	0.760047	fe80::1234:5678:9abc::1	ff02::1	mDNS	511	Standard query response 0x0000 PTR, cache flush DESKTOP-GAIVDV7._dsvnc._tcp.local SRV, cache flush 0 0 7680 DESKTOP-GAIVDV7.local TXT, cache flush A, cache flush 192.168.1.1...
27	0.760057	192.168.1.83	224.0.0.251	mDNS	427	Standard query response 0x0000 SRV, cache flush 0 0 7680 DESKTOP-GAIVDV7.local TXT, cache flush A, cache flush 192.168.1.83 AAAA, cache flush 2000:1700:8af0:9500:131 AAAA, ...
28	0.760938	fe80::1234:5678:9abc::1	ff02::1	mDNS	447	Standard query response 0x0000 SRV, cache flush 0 0 7680 DESKTOP-GAIVDV7.local TXT, cache flush A, cache flush 192.168.1.83 AAAA, cache flush 2000:1700:8af0:9500:131 AAAA, ...
29	2.634367	192.168.1.83	192.168.1.144	TCP	164	56961 → 8009 [PSH, ACK] Seq=1 Ack=1 Win=250 Len=110 [TCP PDU reassembled in 317]
30	2.632709	192.168.1.144	192.168.1.83	TCP	164	8009 → 56961 [PSH, ACK] Seq=1 Ack=111 Win=388 Len=110 [TCP PDU reassembled in 317]
31	2.664771	192.168.1.83	192.168.1.144	TCP	54	56961 → 8009 [ACK] Seq=111 Ack=111 Win=255 Len=0
32	2.690887	2000:1404:a00:5::17	2000:1700:8af0:9500::	IPv6	113	Application Data
33	2.690887	2000:1404:a00:5::17	2000:1700:8af0:9500::	IPv6	98	Application Data
34	2.690887	2000:1404:a00:5::17	2000:1700:8af0:9500::	TCP	74	65473 → 443 [ACK] Seq=64 Ack=5 Win=255 Len=0
35	2.690979	2000:1700:8af0:9500::	2000:1404:a00:5::17	TCP	52	who has 192.168.1.83? Tell 192.168.1.254
36	3.314047	HAPAA_22:6a:81	AzureWaveTec_07:53:1f	ARP	42	192.168.1.83 is at 90:00:00:00:00:00
37	3.314055	AzureWaveTec_07:53:1f	HAPAA_22:6a:81	ARP	107	get-next-request 1.3.6.1.2.1.25.3.5.1.2.1.1
38	4.123728	fe80::1234:5678:9abc::1	ff02::1	mDNS	108	get-response 1.3.6.1.2.1.31.1.5.0
39	4.200040	fe80::1234:5678:9abc::1	ff02::1	mDNS	164	57034 → 8009 [PSH, ACK] Seq=111 Ack=111 Win=252 Len=110 [TCP PDU reassembled in 238]
40	5.409712	192.168.1.83	192.168.1.65	TCP	164	8009 → 57034 [PSH, ACK] Seq=111 Ack=221 Win=434 Len=110 [TCP PDU reassembled in 239]
41	5.409815	192.168.1.65	192.168.1.83	TCP	54	57034 → 8009 [ACK] Seq=221 Ack=221 Win=251 Len=0
42	5.499877	192.168.1.83	192.168.1.65	TCP	86	Neighbor Solicitation for 2000:1700:8af0:9500:131 From 90:00:00:00:00:00
43	7.130666	2000:1700:8af0:9500::	2000:1700:8af0:9500::	ICMPv6	86	Neighbor Solicitation for 2000:1700:8af0:9500:131 From 90:00:00:00:00:00
44	7.130666	2000:1700:8af0:9500::	2000:1700:8af0:9500::	ICMPv6	86	Neighbor Solicitation for 2000:1700:8af0:9500:131 From 90:00:00:00:00:00
45	7.130666	fe80::1234:5678:9abc::1	ff02::1	mDNS	86	Neighbor Solicitation for fe80::1234:5678:9abc::1 From 90:00:00:00:00:00

I decided to take a closer look at the very first packet I captured.

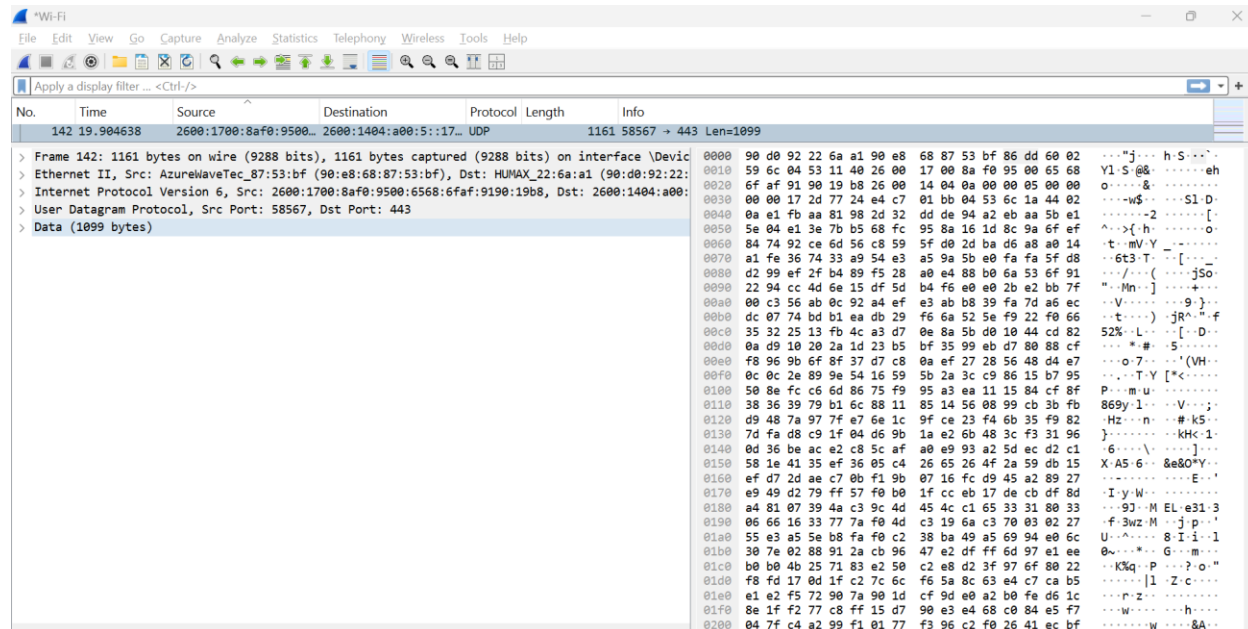


Upon analysis, I determined that the packet used the mDNS protocol, which stands for multicast DNS. The source IP address of this packet was 192.168.1.83, and it was directed to a multicast destination IP of 224.0.0.251. Essentially, this packet indicates that the device at that IP address is announcing its presence on the local network using multicast DNS. This allows other Windows devices on the same local network to share Windows Update data directly with each other, rather than each device downloading updates individually from Microsoft servers.

While this functionality is convenient, it also carries certain security considerations. By advertising its Delivery Optimization service through mDNS, the system reveals some metadata on the local network. Although this behavior is typical for Windows, it could provide an entry point for a malicious actor on the same network to probe for vulnerabilities in the Delivery Optimization service (DoSvc), particularly if the system is not fully patched.

To reduce this risk, users can disable Delivery Optimization in Windows Update settings if the feature is unnecessary. Additionally, on public or untrusted networks, it is advisable to avoid allowing such broadcasts, as sharing this information in less secure environments may increase exposure.

I also analyzed a UDP packet.



I examined Packet 142, which utilized the UDP protocol and was directed to port 443, a port commonly associated with HTTPS traffic. This packet was an Internet Protocol Version 6 (IPv6) UDP datagram carrying approximately 1,099 bytes of data from the local host to an external server. From a security perspective, this type of traffic is generally expected during normal web browsing or when using modern applications.

That said, it is important to recognize that attackers could theoretically exploit this kind of UDP traffic for covert communication channels or data exfiltration. While such misuse is unlikely under typical conditions, the potential risk highlights the importance of monitoring for unusually large volumes of UDP/443 traffic directed toward unexpected or suspicious destinations.

Question 3: What did I learn?

This exercise helped me understand the types of network traffic present in a residential setting and the sheer volume of data that can be captured in a short time frame, showing me that every click, page load, and background service generates multiple exchanges across different protocols. It highlights how much sensitive information constantly moves across a network, often unnoticed. This reinforces the importance of cybersecurity practices such as traffic monitoring, protocol analysis, and intrusion detection, since attackers could easily hide malicious activity within the overwhelming volume of normal packets. Wireshark not only revealed the scale of background communication but also showed me why continuous monitoring and filtering are critical for protecting networks.

PHISHING

Question 1: what did I do?

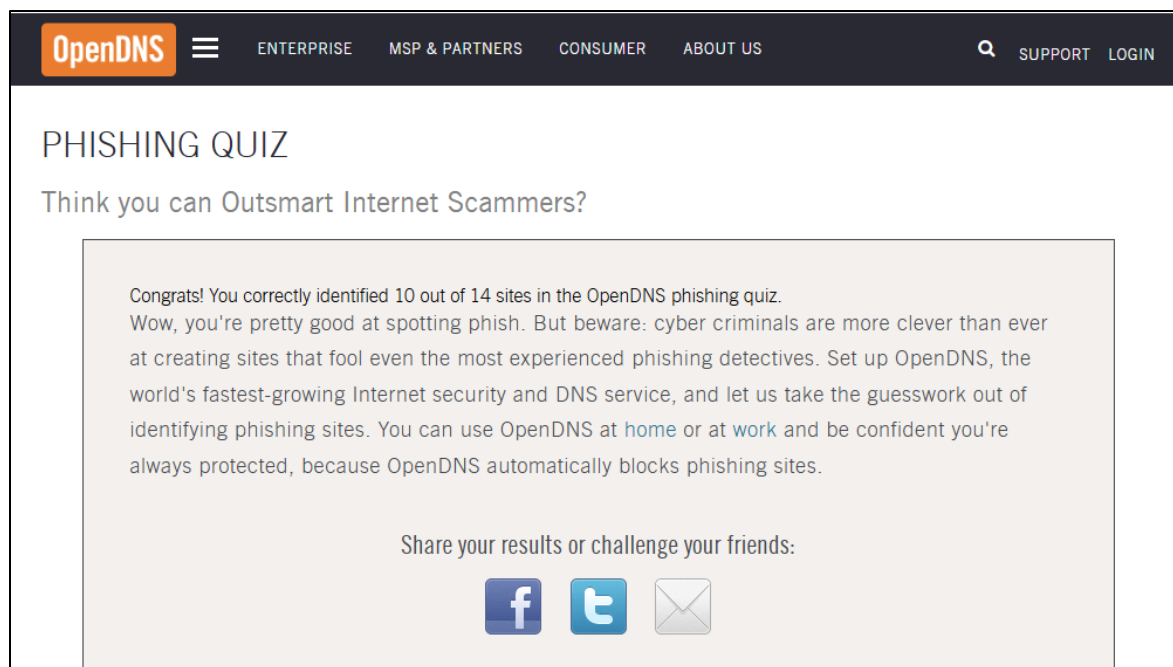
As part of this assignment, I completed a phishing awareness quiz on OpenDNS, a cloud-based DNS security service, and another on SonicWall, a network security company recognized for its firewalls, VPN solutions, and email protection tools.

OpenDNS operates by intercepting DNS queries when a user enters a website address. It translates the domain name into an IP address while routing the request through its own servers, where it compares the domain against a large database of safe, suspicious, and malicious sites. If the site is deemed safe, access proceeds as normal; if it is malicious, the request is blocked, and a warning page is displayed.

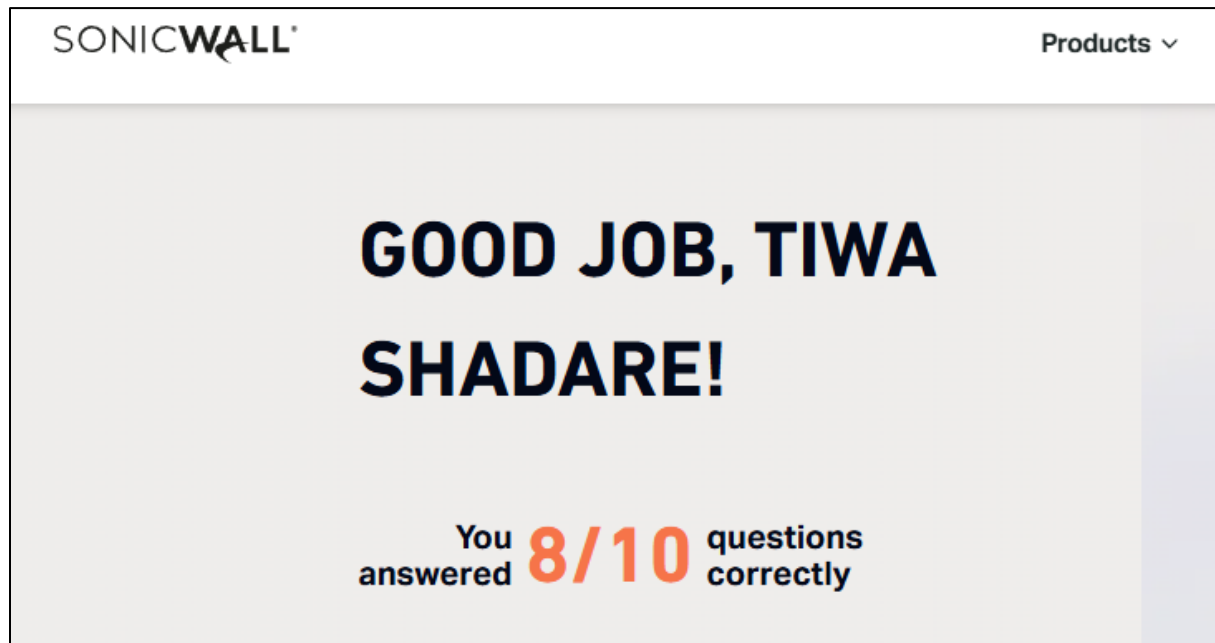
SonicWall, by contrast, secures networks at a deeper level. Its firewalls perform deep packet inspection on all inbound and outbound traffic to identify and block threats such as viruses, intrusions, and malware. Additionally, it provides email filtering to stop phishing attempts, spam, and malicious attachments before they reach users' inboxes. SonicWall's cloud-based threat intelligence systems also continuously update firewall signatures and detection engines in real time, ensuring protection against emerging threats.

Question 2: what were the results?

On the OpenDNS quiz, I successfully identified 10 out of 14 sites correctly as either legitimate or phishing attempts. This really emphasized how even when you're actively trying to spot scams, it's still possible to miss a few.



On the SonicWall quiz, I did a bit better, correctly identifying 8 out of 10 scenarios. This experience highlighted how challenging it can be for the average person who might not be on high alert to recognize phishing attempts.



Question 3: What did I learn?

It really drives home the importance of strengthening cybersecurity awareness so that everyone is better equipped to spot suspicious emails or URLs. From this exercise, I learned that detecting phishing is much harder in practice than it seems, even when you know what to look for. My results showed that attackers use subtle tactics to mimic legitimate sites and emails, which can trick even vigilant users. This reinforced the importance of continuous cybersecurity awareness training and highlighted that technology tools (like OpenDNS and SonicWall) should complement human judgment. Ultimately, effective protection depends on a layered approach that combines informed users with robust security tools.